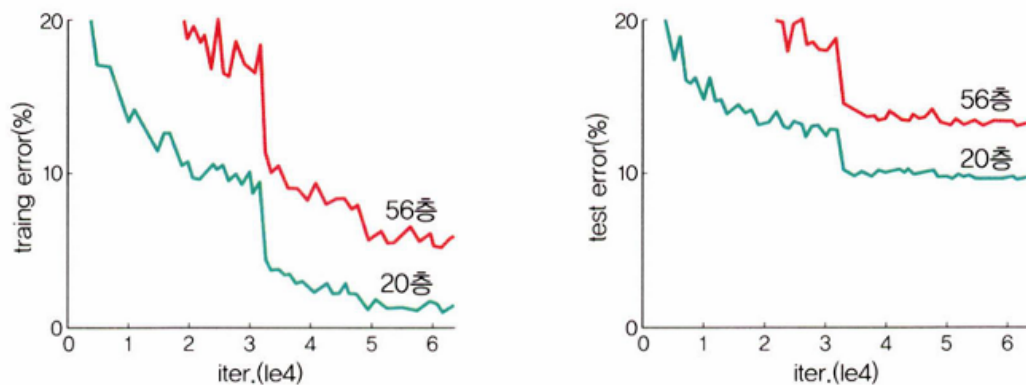


6장_합성곱 신경망 2_Part 2(6.1.5~)

6.1.5. ResNet

- "Deep Residual Learning for Image Recognition" - MS
- **깊어진 신경망**을 효과적으로 학습하기 위한 방법으로 **레지듀얼(residual)** 개념 고안
 - **깊어진 신경망**: 신경망은 깊이가 깊어질수록 성능이 좋아지다가 일정한 단계에 다르면 오히려 성능이 나빠짐

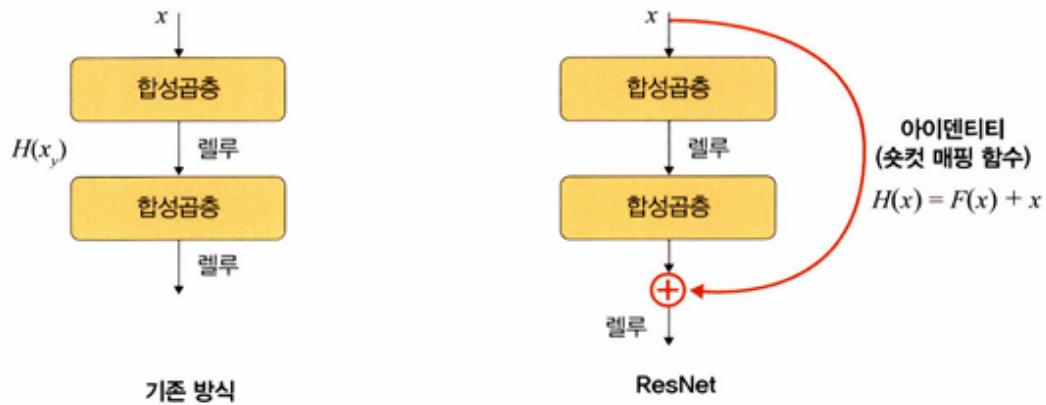
♥ 그림 6-24 네트워크 56층이 20층보다 더 나쁜 성능을 보임



x: iter y: loss

- **residual block**: 기울기가 잘 전파될 수 있도록 일종의 숏컷(shortcut, skip connection)을 만드는 것
GoogLeNet(2014) - 22층 / ResNet - 152층 → 기울기 소멸 문제 발생 가능

▼ 그림 6-25 ResNet 구조



ResNet 구조

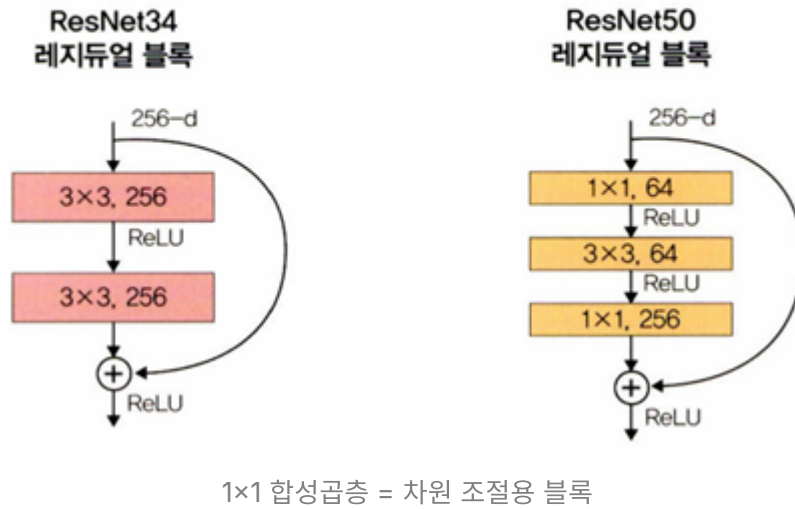
- 블록 = 계층(레이어)의 묶음
- residual block: 하나의 숏컷을 포함한 레이어 묶음
- ResNet: 여러 Residual block을 여러 개 쌓은 것

▼ 그림 6-26 ResNet 모델 전체 네트워크



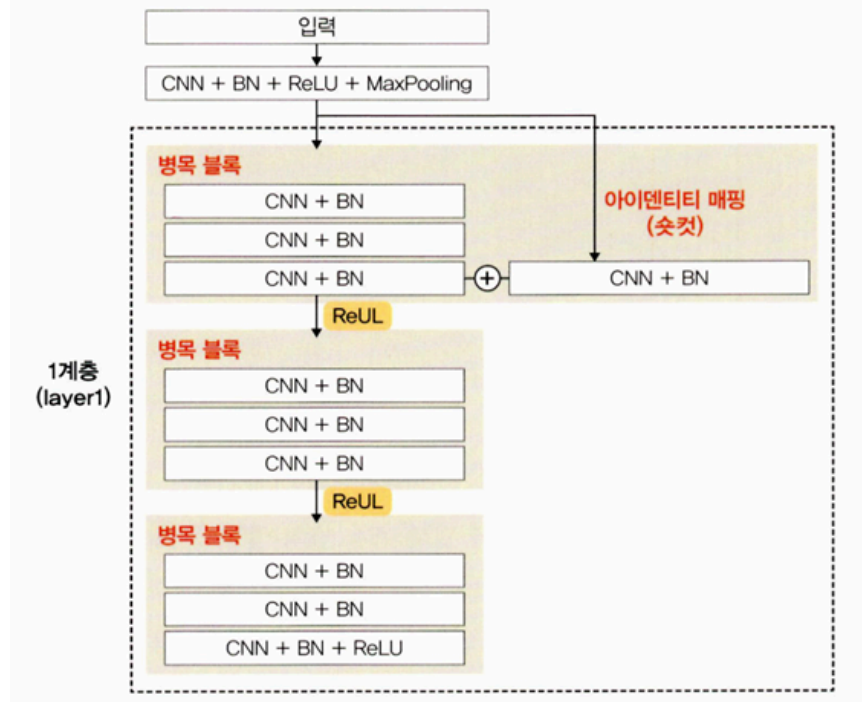
- 병목 블록(bottleneck block): 계층의 깊이가 깊어질수록 파라미터가 무제한으로 커지는 문제를 해결하기 위한 블록. 3x3 합성곱층 앞뒤로 1x1 합성곱층을 붙여 **1x1 합성곱층의 채널 수를 조절**하면서 차원을 줄였다 늘리는 것이 가능해짐.

▼ 그림 6-27 기본 블록과 병목 블록



- ResNet34: 합성곱층 32개, 기본 블록(basic block) 16개로 구성됨 → 파라미터 수 39.3216M
- ResNet50: 합성곱층 50개, 병목 블록 사용 → 파라미터 수 6.9632M
- 아이덴티티 매핑(identity mapping, shortcut, skip connection): 입력 x 가 어떤 함수를 통과하더라도 다시 x 라는 형태로 출력되도록 함

▼ 그림 6-28 아이덴티티 매핑(숏컷)



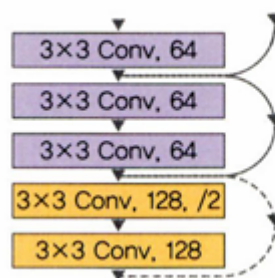
```
def forward(self, x):
    i = x
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2(x)
    x = self.bn2(x)

    if self.downsample is not None:
        i = self.downsample(i) ----- 다운샘플 적용

    x += i ----- 아이덴티티 매핑 적용
    x = self.relu(x)
    return x
```

- 입력 x (28, 28, 64) → 변수 i에 저장
- x는 합성곱층을 통과 후 i가 더해짐
⇒ (텐서끼리 더해주려면 크기가 같아야 하므로 1x1 Conv에서 차원을 맞춰줘야 함)
출력되는 피쳐 맵의 최종 크기(차원)도 (28, 28, 64)
- 다운샘플(downsample): 특성 맵 크기를 줄이기 위한 것 ≡ 풀링

▼ 그림 6-29 ResNet 네트워크의 일부

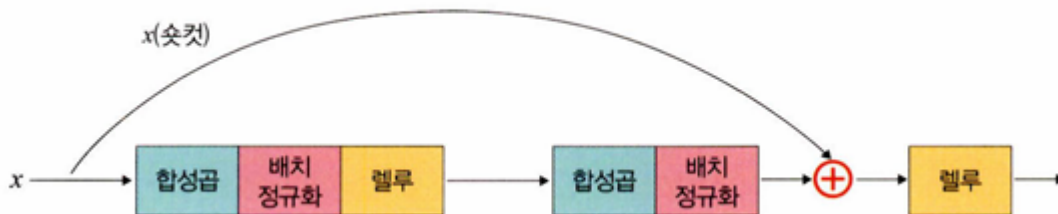


- 보라색 블록의 특성 맵 크기: (28, 28, 64)
- 노란색 영역 → 채널 수가 128로 늘어남 & 합성곱층 스트라이드가 2로 늘어남: (14, 14, 128)
- 보라색이랑 노란색 형태가 다름

⇒ 아이덴티티 매핑을 위한 다운샘플 필요 (위 그림 점선): 스트라이드 2인 1x1 합성곱층을 하나 연결해 주면 됨

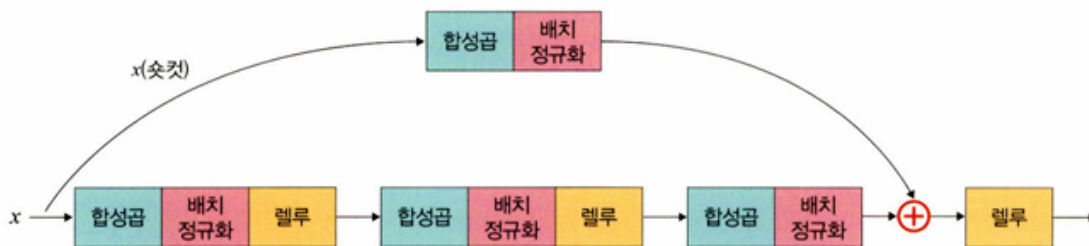
- 아이덴티티 블록: 입력과 출력의 차원이 같은 블록

▼ 그림 6-31 아이덴티티 블록



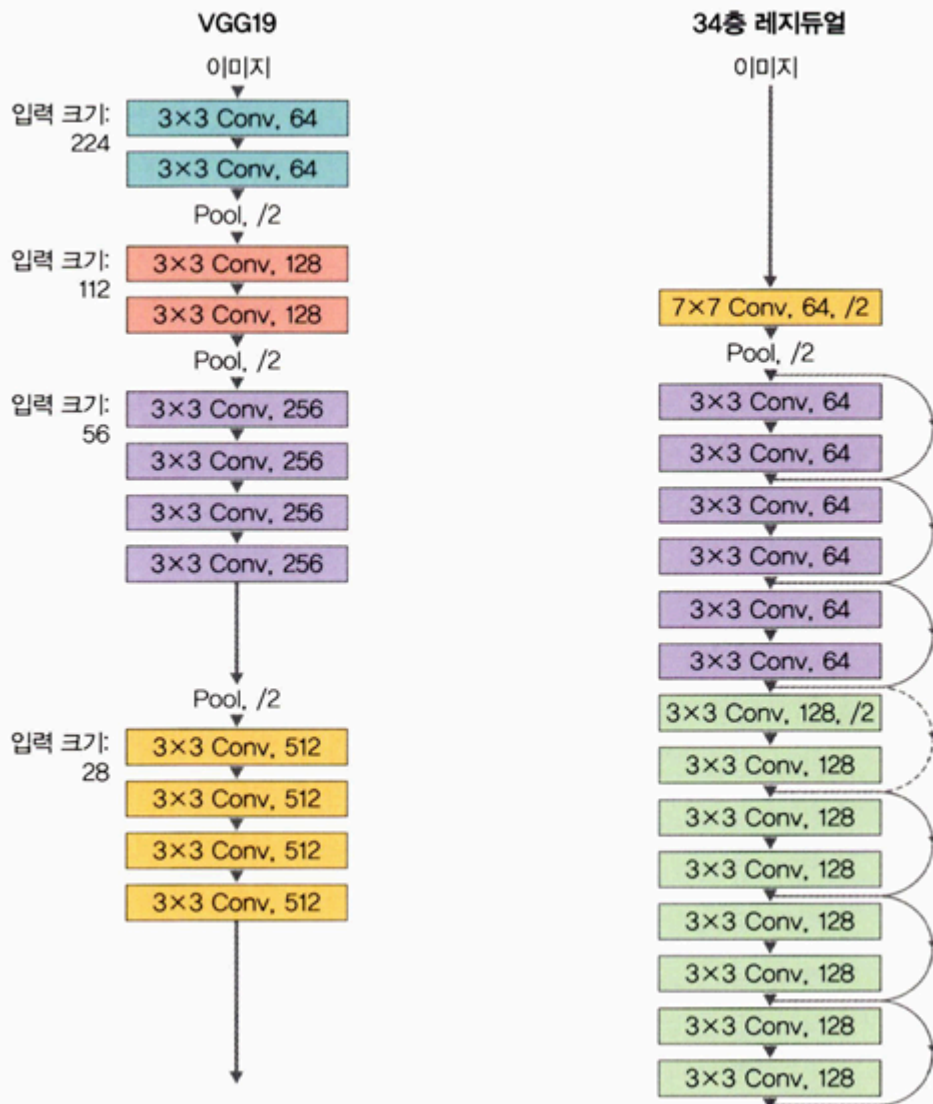
- projection-shortcut, 합성곱 블록: 입력 및 출력 차원이 동일하지 않아 입력의 차원을 출력에 맞추어 변경해야 하는 블록

▼ 그림 6-30 합성곱 블록



✅ ResNet = VGG19 구조 + 합성곱층 추가 (깊이 ↑) + 숏컷 추가

♥ 그림 6-32 VGG19와 ResNet 비교



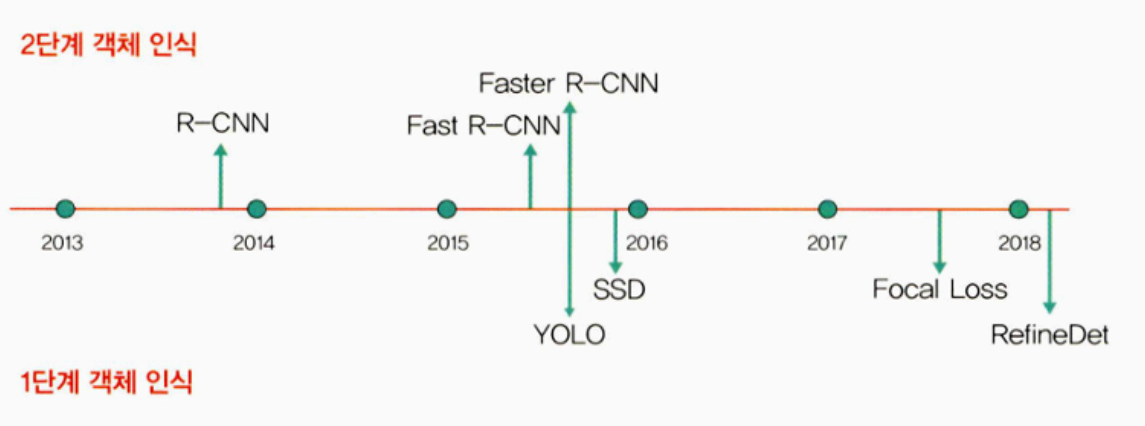
ResNet 구현

6.2. 객체 인식을 위한 신경망

객체 인식(object detection): 여러 가지 객체에 대한 분류 + 객체의 위치 정보를 파악하는 위치 검출 (bounding box & localization)

- 딥러닝을 이용한 객체 인식 알고리즘

▼ 그림 6-35 1단계 객체 인식 vs 2단계 객체 인식 흐름도



- 1단계 객체 인식(1-stage detector): 분류 & 위치 검출을 동시에 행하는 방법
 - 빠르지만 정확도가 낮음
 - YOLO(You Only Look Once) 계열, SSD 계열
- 2단계 객체 인식(2-stage detector): 분류 & 위치 검출을 순차적으로 행하는 방법
 - 느리지만 정확도가 높음
 - R-CNN 계열
- 객체 인식 활용: 자율 주행 자동차, CCTV, 무인 점포 등

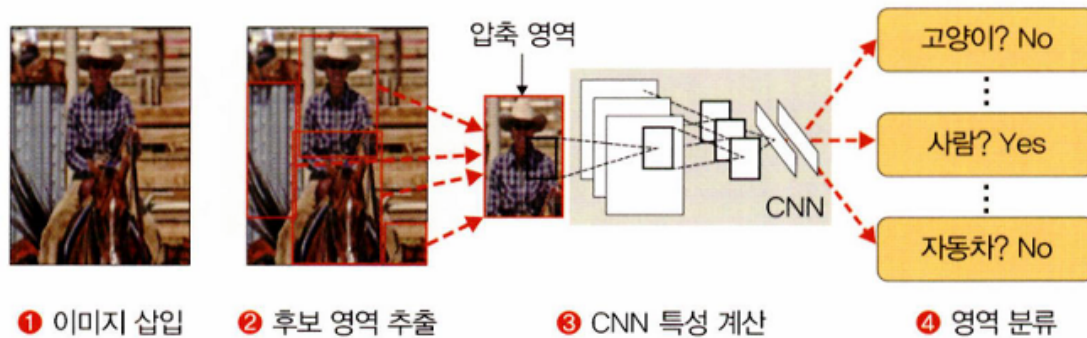
6.2.1. R-CNN

- 예전의 객체 인식 알고리즘: 슬라이딩 윈도우(sliding window) 방식
 - 이미지 왼쪽 위부터 일정 크기의 경계 상자를 만들고, 그 안에서 객체를 탐색하는 과정을 반복
 - 일정한 크기를 가지는 윈도우를 가지고 이미지의 모든 영역을 탐색하며 객체를 검출해 내는 방식
 - 알고리즘의 비효율성 때문에 많이 사용하지 않음
- 현재 객체 인식 알고리즘: 선택적 탐색(selective search) 알고리즘을 적용한 후보 영역(region proposal)
 - 후보 영역: 영상/이미지에서 객체가 있을 법한 영역

R-CNN (Region-based CNN): 이미지 분류를 수행하는 CNN + 후보 영역 알고리즘 결합한 알고리즘

- R-CNN 수행 과정

▼ 그림 6-36 R-CNN 학습 절차



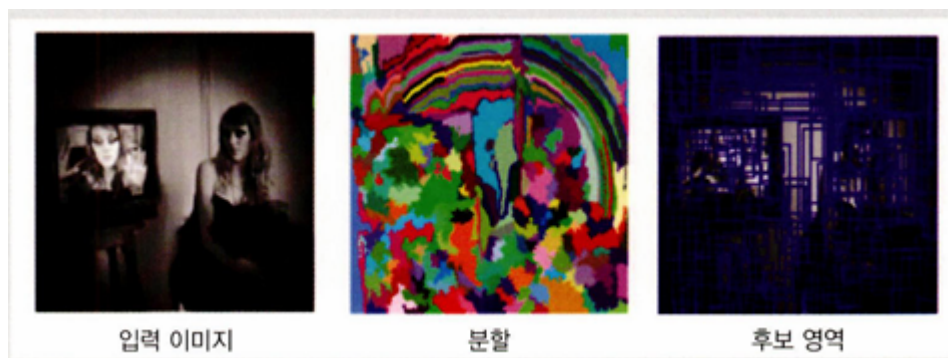
1. 이미지 입력
2. 2000개의 바운딩 박스(bounding box)를 선택적 탐색 알고리즘으로 추출한 뒤 잘라내고(cropping) CNN 모델에 넣기 위해 같은 크기(227x227 px)로 통일(warping)
3. 크기가 동일한 이미지 2000개에 각각 CNN 모델 적용
4. 각각 분류 → 결과 도출

▼ 선택적 탐색

- 객체 인식이나 검출을 위한 가능한 후보 영역을 알아내는 방법
- 분할 방식을 이용해 시드(seed)선정 → 그 시드에 대한 완전 탐색 적용
- 선택적 탐색 과정

1. 초기 영역 생성 (sub-segmentation)

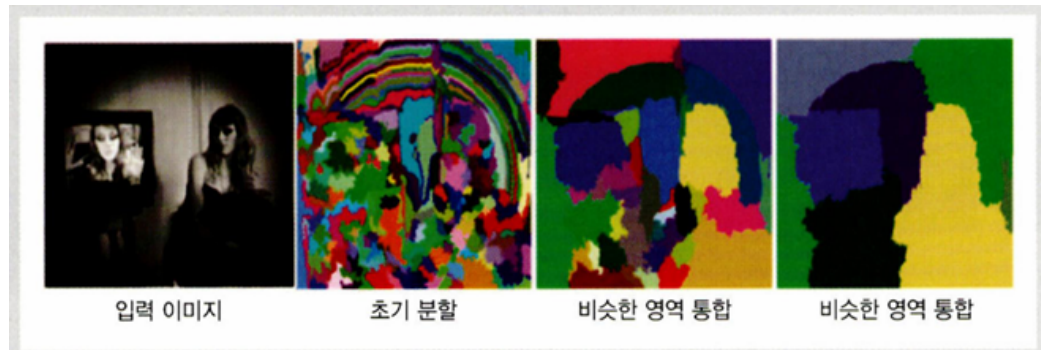
각각의 객체가 영역 한 개에 할당될 수 있도록 많은 초기 영역 생성
입력된 이미지를 영역 다수 개로 분할



2. 작은 영역 통합

1단계에서 나눈 영역들을 비슷한 영역으로 통합

그리디 알고리즘을 사용해 비슷한 영역이 하나도 통합될 때까지 반복



3. 후보 영역 생성

2단계에서 통합된 이미지들을 기반으로 후보 영역(바운딩 박스) 추출



- 완전 탐색(exhaustive search): 후보가 될 만한 대상의 크기 및 비율이 모두 다른 상황을 고려해 후보 영역을 찾는 기법
 - 분할(segmentation): 영상 데이터의 특성(색상, 모양, 무늬 등)에 따라 분할하여 후보 영역 선정하는 기법
 - 후보 영역(바운딩 박스): 3D 객체의 형태를 모두 포함할 수 있는 최소 크기의 박스
 - 시드(seed): 영상에서는 특정 기준점의 픽셀 → 점점 의미가 같은 영상 범위까지 픽셀을 확장해 나가면서 분할하는데, 이때 특정 기준점이 되는 픽셀
- 단점
 - 선택적 탐색 3단계의 복잡한 학습 과정
 - 긴 학습 시간, 대용량 저장 공간 요구
 - 객체 검출 속도 문제

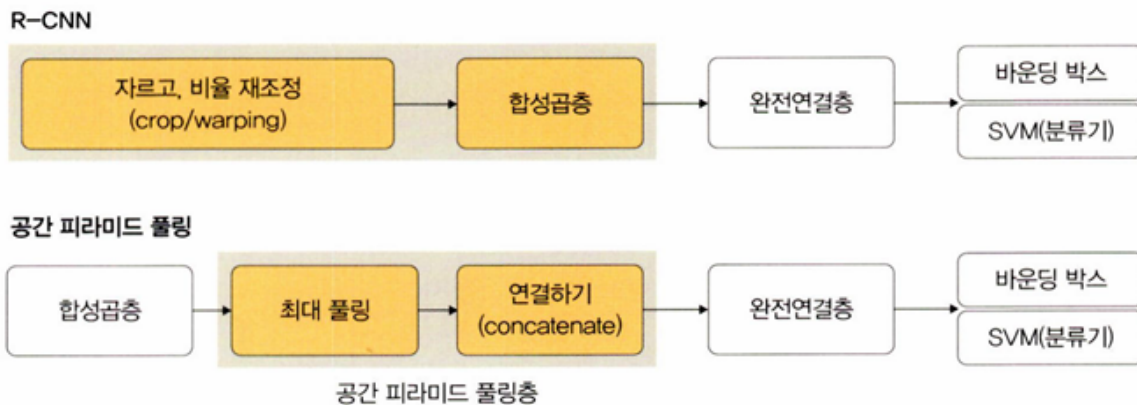
→ Fast R-CNN

6.2.2. 공간 피라미드 풀링

- 기존 CNN 구조(ex.R-CNN): 완전연결층을 위해 입력 이미지를 고정해야 해서, 이미지를 고정된 크기로 자르거나(crop) 비율을 조정(warp)해야 함 → **물체의 일부분이 잘리거나 본래의 생김새와 달라지는 문제점**

공간 피라미드 풀링 (spatial pyramid pooling)

▼ 그림 6-40 공간 피라미드 풀링



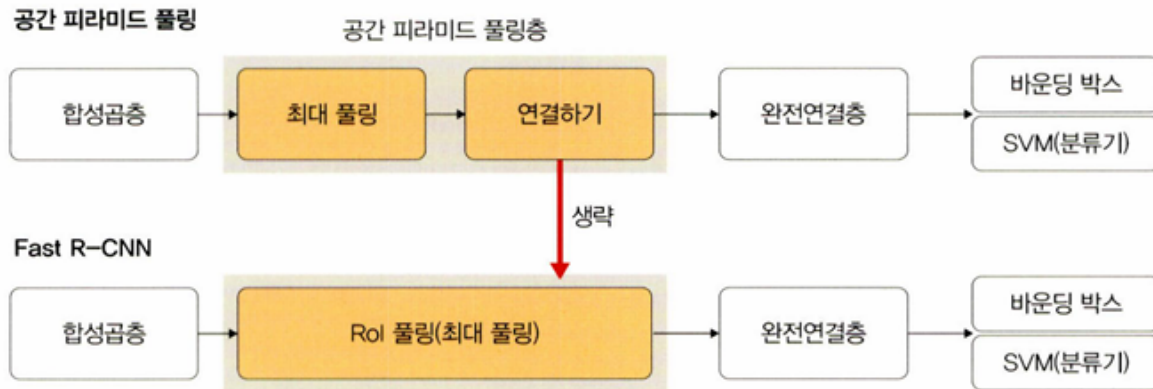
입력 이미지의 크기에 관계없이 합성곱층을 통과시키고, 완전연결층에 전달되기 전에 특성 맵들을 동일한 크기로 조절해 주는 풀링층을 적용하는 기법

- 원본 이미지의 특징이 훼손되지 않는 특성 맵 얻을 수 있음
- 이미지 분류나 객체 인식 등 여러 작업에 적용 가능

6.2.3. Fast R-CNN

- R-CNN 문제점: 바운딩 박스마다 CNN을 돌리고 분류해서 **학습 시간이 길어짐**

Fast R-CNN



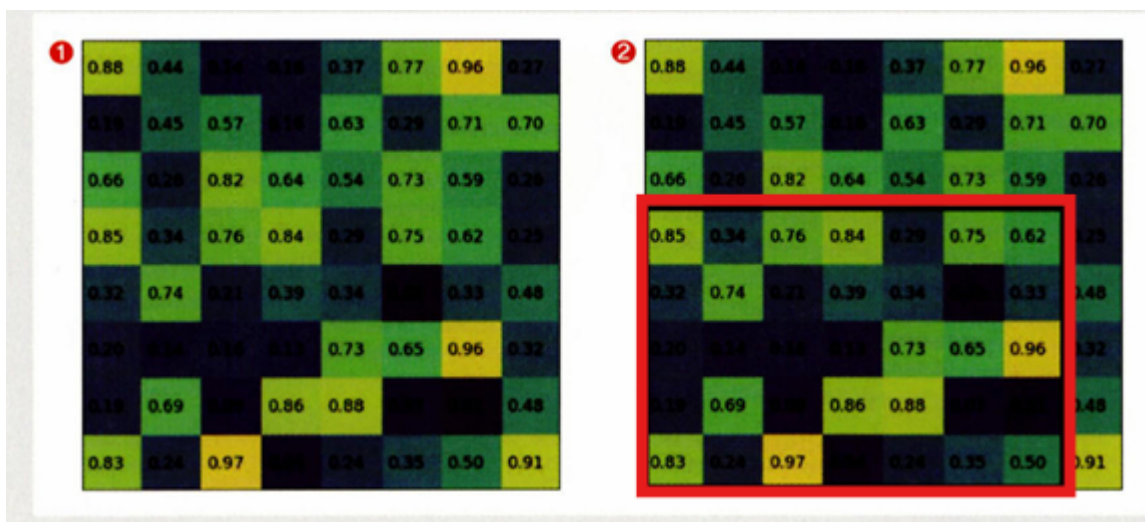
→ **RoI 풀링** 도입: 선택적 탐색에서 찾은 바운딩 박스 정보가 CNN을 통과하며 유지되도록 하고, 최종 CNN 특성 맵은 풀링을 적용해 완전연결층을 통과하도록 크기 조정

⇒ 바운딩 박스마다 CNN 돌리는 시간 단축

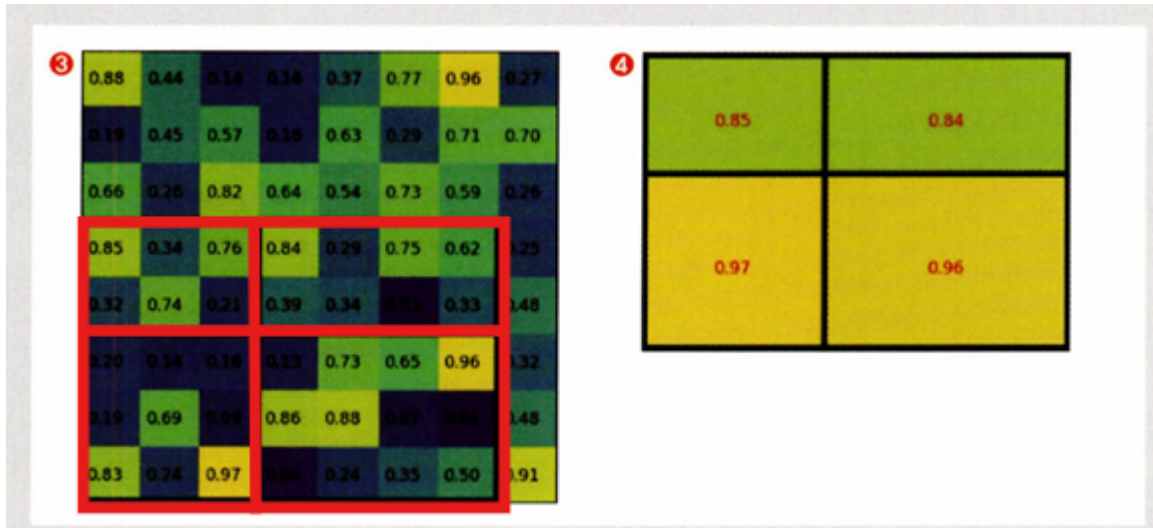
(공간 피라미드 풀링은 최대 풀링을 전부 적용한 뒤 완전연결층을 위한 풀링층이 따로 있는 것, RoI 풀링은 풀링 적용하면서 크기를 맞춤형으로 축소함)

▼ RoI 풀링

크기가 다른 특성 맵의 영역마다 스트라이드를 다르게 최대 풀링을 적용해 결괏값 크기를 동일하게 맞추는 방법



(1) 8×8 특성 맵 (2) 선택적 탐색으로 뽑은 7×5 크기의 후보 영역



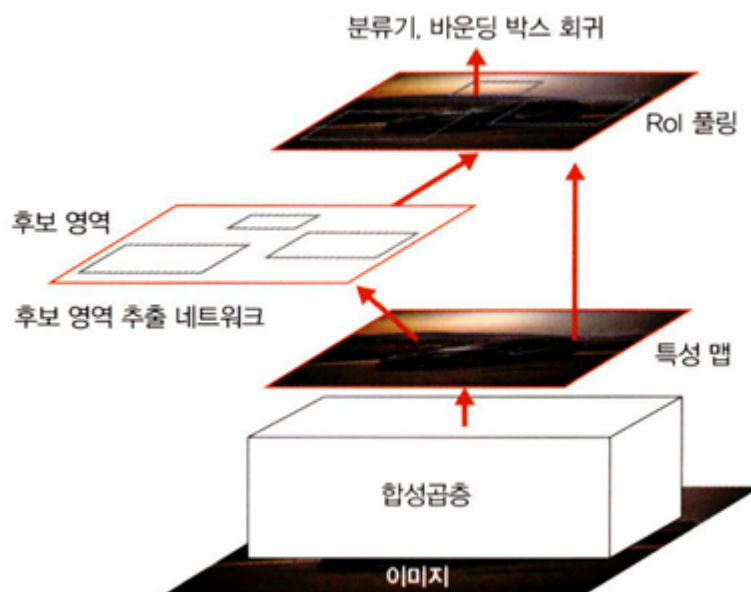
(3) 후보 영역을 2x2로 만들기 위해 스트라이드(7/2=3, 5/2=2)로 풀링 영역 선정 (4) 최대 풀링 적용

6.2.4. Faster R-CNN

= Fast R-CNN + RPN(Region Proposal Network, 후보 영역 추출 네트워크)

- 후보 영역 생성을 CNN 내부 네트워크에서 진행할 수 있도록 설계함
- 외부의 느린 선택적 탐색(cpu) 대신 내부의 빠른 RPN(gpu) 사용

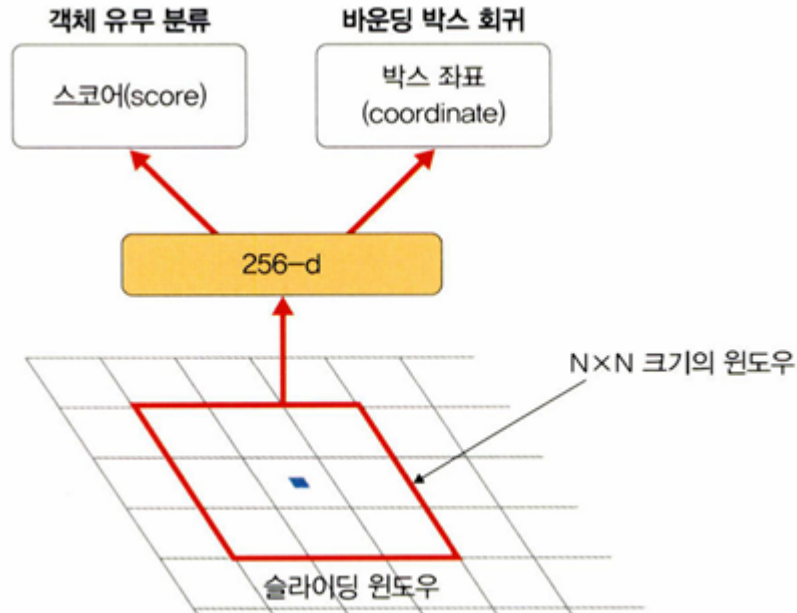
▼ 그림 6-43 Faster R-CNN



- 구조: 마지막 합성곱층 → RPN → RoI 풀링, 분류기(classifier), 바운딩 박스 회귀(bounding-box regression)

- 바운딩 박스 회귀: 바운딩 박스가 이미지의 객체를 정확히 포착해 포함시킬 수 있도록 조정

- 후보 영역 추출 네트워크(RPN)



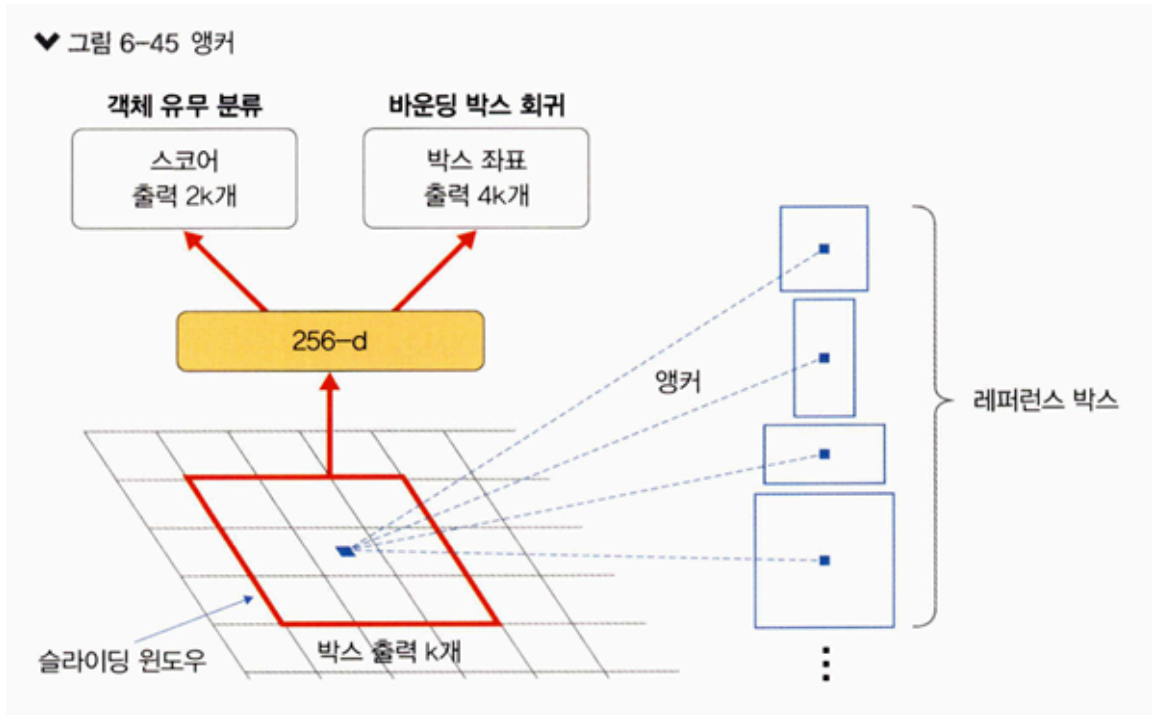
- **객체 유무 분류**: 특성 맵 안에 $N \times N$ 크기 윈도우를 입력으로 받고, 해당 영역에 객체 존재 유무를 판단하기 위해 이진 분류(binary classification)를 수행하는 작은 네트워크 생성

- **바운딩 박스 회귀**: 위치 보정(좌표점 추론)

→ 하나의 특성 맵에 있는 모든 영역에 대한 객체 존재 유무 확인을 위해 슬라이딩 윈도우 방식으로 객체 탐색

⚠ 고정된 $N \times N$ 크기의 입력만으로는 다양한 크기와 비율을 가진 이미지 내 객체들을 수용하기 어려움

- 앵커(anchor)



- 여러 크기와 비율의 레퍼런스 박스(reference box) k개를 미리 정의하고 각각의 슬라이딩 윈도우 위치마다 박스 k개를 출력하도록 설계하는 방식
 - 후보 영역 추출 네트워크의 출력 값
 - 모든 앵커 위치에 대해 각각 객체와 배경을 판단하는 2k개의 **분류에 대한 출력**
 - x, y, w, h 위치 보정 값을 위한 4k개의 **회귀 출력**
- w x h 크기 특성 맵 - w x h x k 개의 앵커 존재

6.3. 이미지 분할을 위한 신경망

이미지 분할 (image segmentation): 신경망을 훈련시켜 이미지를 픽셀 단위로 분할하는 것

→ 픽셀 단위로 객체 추출

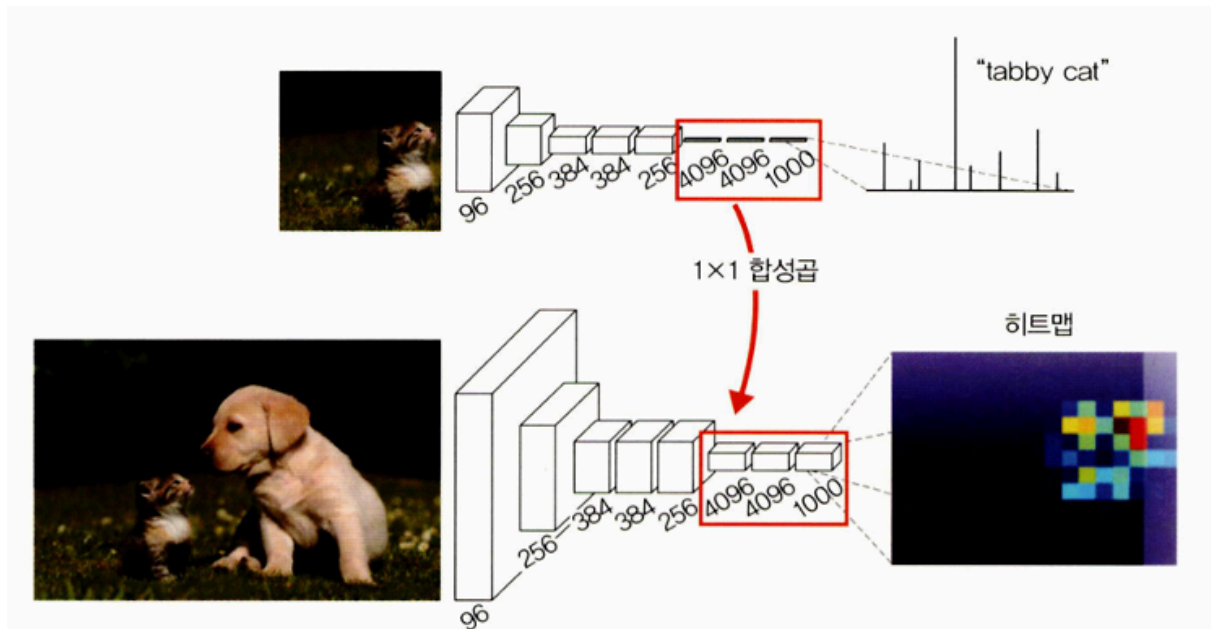
- 대표 네트워크: 완전 합성곱 네트워크(FCN), 합성곱 & 역합성곱 네트워크, U-Net, PSPNet, DeepLabv3/DeepLabv3+

6.3.1. 완전 합성곱 네트워크

- 완전연결층의 한계: 고정된 크기의 입력만 받아들이며, 완전연결층을 거친 후에는 위치 정보가 사라짐

완전 합성곱 네트워크 (Fully Convolutional Network, FCN): 완전연결층 → 1x1 Conv 대체

⇒ CNN 기반 모델(AlexNet, VGG16, GoogLeNet)을 변형시켜 이미지 분할에 적합하도록 만든 네트워크

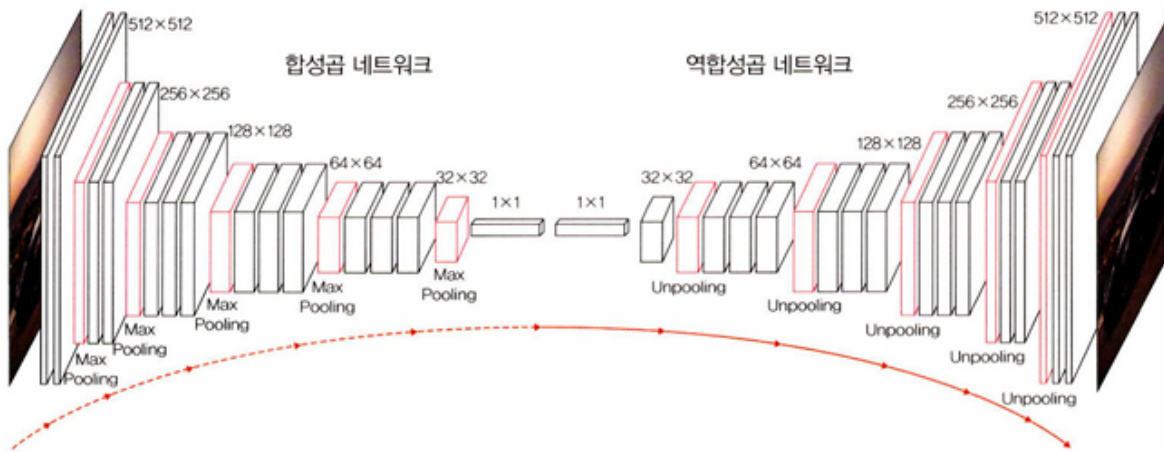


- AlexNet 아래쪽에서 사용된 완전연결층 세 개를 1x1 Conv로 변환
→ 위치 정보가 남아 있어서 히트맵(heatmap) 그림 등으로 고양이 위치 확인 가능
(합성곱은 커널 쓰니까)
- heatmap: 색상으로 표현할 수 있는 다양한 정보 → 이미지 위에 열 분포 형태의 그래픽으로 출력

6.3.2. 합성곱 & 역합성곱 네트워크

- 완전 합성곱 네트워크의 단점
 - 여러 단계의 합성곱층&풀링층 → 해상도 ↓
 - 낮아진 해상도 복원을 위해 업샘플링 방식 사용 → 이미지의 세부 정보들을 잃어버림

합성곱 & 역합성곱 네트워크 (convolutional & deconvolutional network)

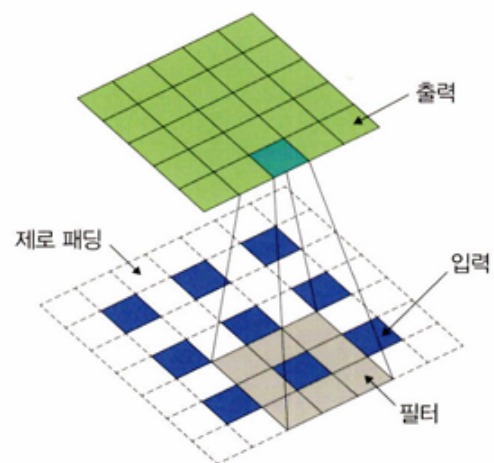


- 역합성곱: 합성곱층과 반대로 합성곱 연산으로 특성 맵 크기를 증가시키는 것
 - 입력 이미지와 같은 크기로 만들기 위해 사용
 - 시맨틱 분할 등에 활용 가능
 - 업샘플링이라고도 함

• 역합성곱 동작

1. 각 픽셀 주위에 제로 패딩 추가
2. 합성곱 연산 수행

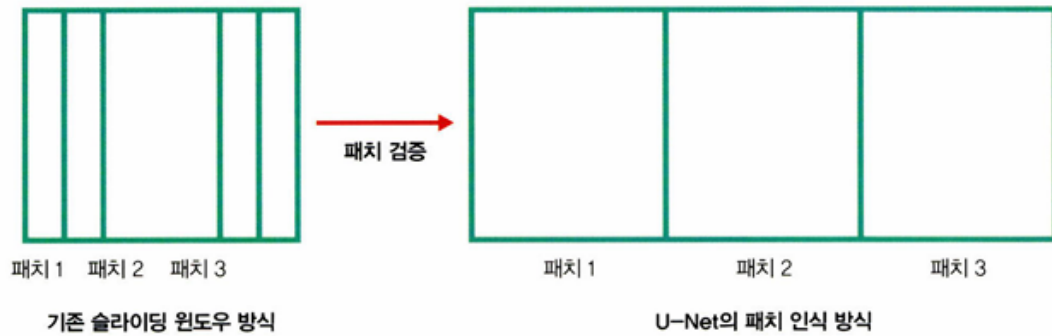
▼ 그림 6-48 역합성곱 진행 방식



6.3.3. U-Net

- 바이오 메디컬 이미지 분할을 위한 합성곱 신경망
- 특징
 - 속도가 빠름: 이미 검증이 끝난 패치(patch - 이미지 인식 단위)는 건너뛰

▼ 그림 6-49 슬라이딩 윈도우 방식



기존 슬라이딩 윈도우 방식은 이전 패치에서 검증이 끝난 부분을 다음 패치에서 또 검증해서 속도가 느렸음

○ 컨텍스트 인식&지역화 트레이드 오프 문제 개선

- 패치 크기  → 넓은 이미지를 인식하는 데 뛰어남, 컨텍스트(context) 인식에 탁월

but 너무 넓은 범위를 한 번에 인식해서 지역화에는 약함

▼ 지역화(localization)

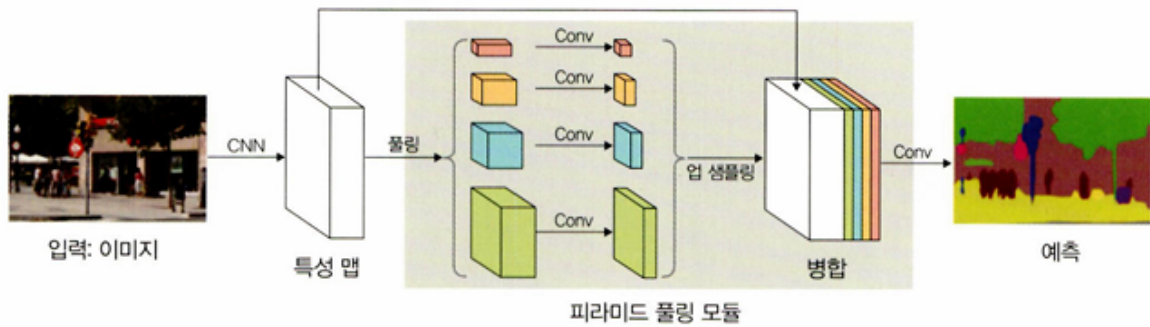
이미지 안에 객체 위치 정보를 출력해 주는 것 - 주로 바운딩 박스 사용

바운딩 박스의 left top, right bottom 좌표 출력



U-Net 구조

♥ 그림 6-52 PSPNet



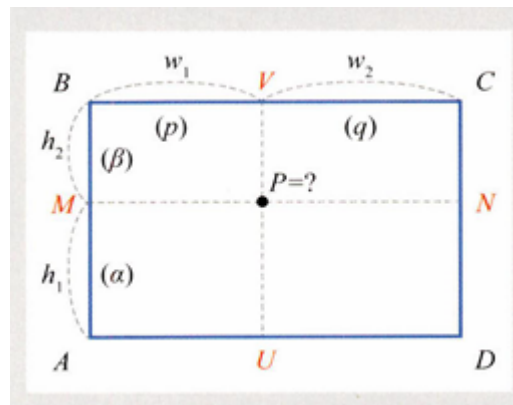
풀링층 개수는 구현에 따라 다르게 설정할 수 있음

PSPNet 훈련 과정

1. 이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링 - 1x1, 2x2, 3x3, 6x6
 - 1x1 크기 특성 맵: 가장 광범위한 정보를 담음
 - 각각 다른 크기의 특성 맵: 서로 다른 영역들의 정보를 담음
2. 1x1 합성곱으로 채널 수 조정 - 출력 채널 수 = 입력 채널 수 / N (풀링층 개수)
3. 모듈 입력 크기에 맞게 특성 맵 업샘플링 - 양선형 보간법(bilinear interpolation) 사용

▼ 양선형 보간법

- 보간법(interpolation): 화소 값을 할당받지 못한 영상(ex. 빈공간)의 품질은 안 좋을 수밖에 없음 → 빈 화소에 값을 할당해 좋은 품질의 영상을 만드는 방법
 - 선형 보간법(linear interpolation): 원시 영상의 화소 값 두 개를 사용해 원하는 좌표의 새로운 화소 값을 계산하는 방법
 - 양선형 보간법(bilinear interpolation): 화소당 선형 보간 세 번 수행(x , y , $x \& y$) → 새롭게 생성된 화소는 가장 가까운 화소 네 개에 가중치를 곱한 값을 합해서 얻음



$$P = q(\beta A + \alpha B) + p(\beta D + \alpha C) \\ = q\beta A + q\alpha B + p\beta D + p\alpha C$$

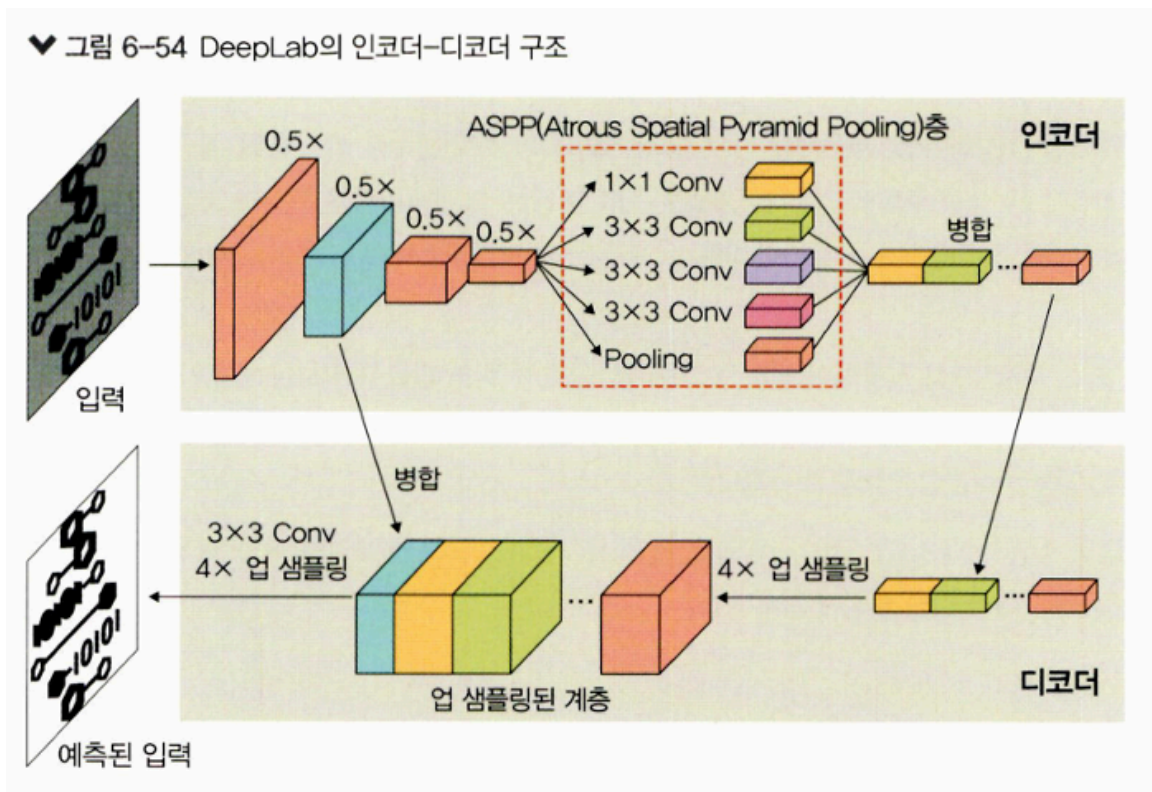
- $a = h1/(h1+h2)$, $b = h2/(h1+h2)$
- $p = w1/(w1+w2)$, $q = w2/(w1+w2)$

4. 원래 특성 맵과 1~3 과정에서 생성한 새로운 특성 맵 병합

6.3.5. DeepLabv3/DeepLabv3+

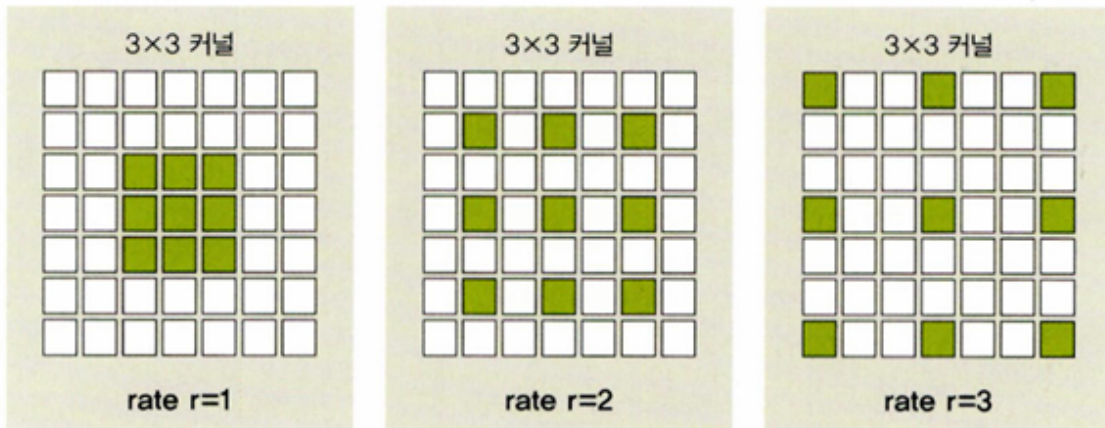
완전연결층 단점 보완을 위해 **Atrous 합성곱**을 사용하는 네트워크

- 인코더-디코더 구조 + Atrous 합성곱 → 인코더에서 추출된 특성 맵 해상도 제어



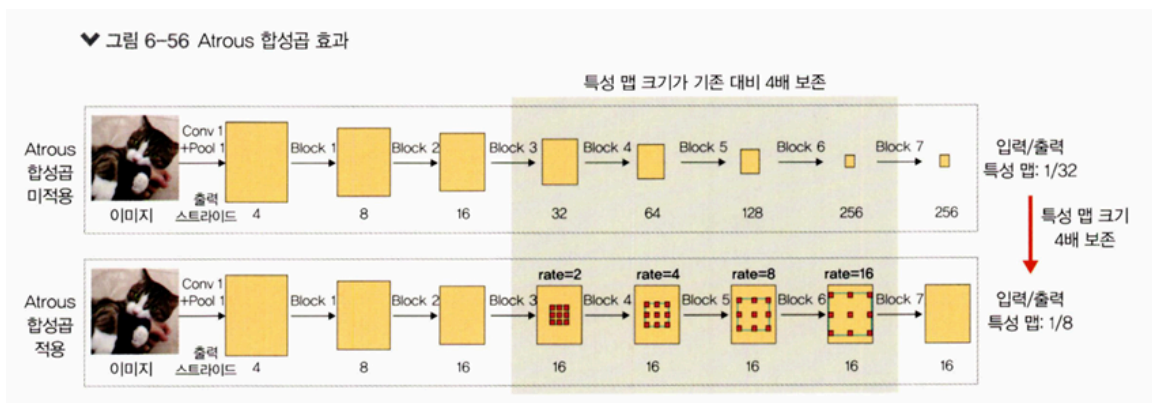
- **Atrous 합성곱**: 필터 내부에 빈 공간을 둔 채로 작동
 - rate 파라미터: 얼마나 많은 빈 공간을 가질지 결정

▼ 그림 6-55 Atrous 합성곱



- 효과: (빈 공간이니까) 파라미터 수를 늘리지 않으면서도 수용 영역(receptive field)을 확대해 특성을 찾는 범위를 넓혀줌 (더 자세하게 들여다 보면서 특성을 찾는다는 것)

→ 이미지 분할 분야에서 많이 사용



▼ 수용 영역(receptive field)

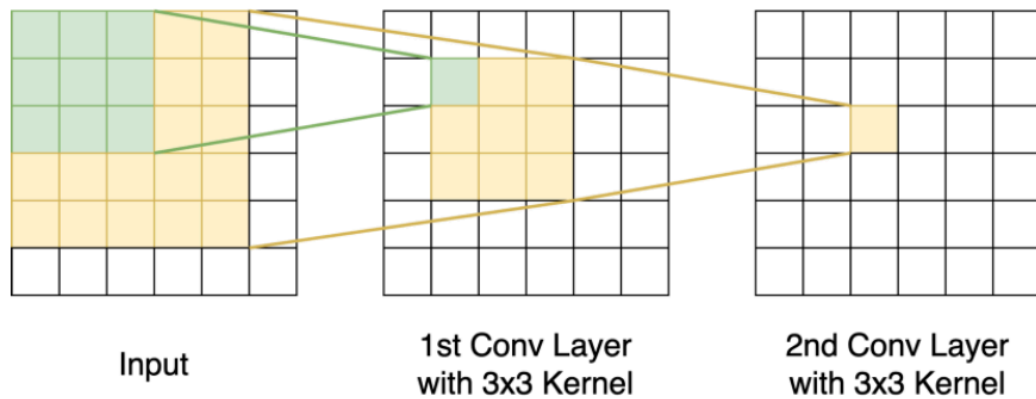


Figure 1: The receptive field in a convolutional neural network with two 3×3 convolutional (conv) layers. In the 2nd conv layer, every pixel has a 5×5 field of view, a.k.a. receptive field.

input space에서 특정 레이어의 피처에 영향을 미치는 범위

- 위 그림에서 *1st Conv Layer*의 초록색 피처에 영향을 주는 건 *Input*의 초록색 3x3 픽셀 범위
- *2nd Conv Layer*의 노란색 피처에 영향을 주는 건 *Input*의 노란색 5x5 픽셀 범위
(합성곱 역으로 따라가다보면)

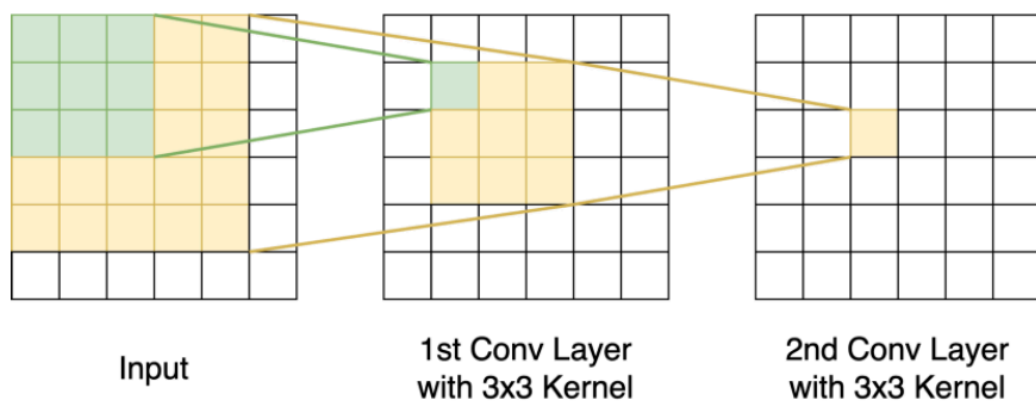


Figure 1: The receptive field in a convolutional neural network with two 3×3 convolutional (conv) layers. In the 2nd conv layer, every pixel has a 5×5 field of view, a.k.a. receptive field.